

Name: Aditi M

USN: 1RUA25BCA0006

Subject: Database Management System

SQL Operators — Theory

Objective

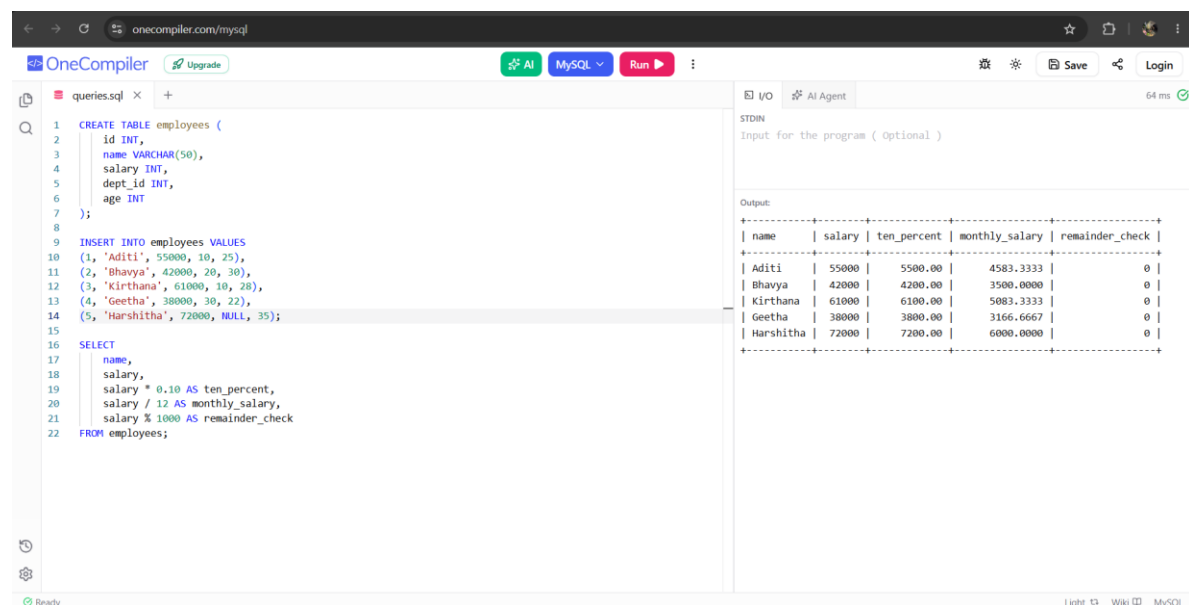
To understand and describe the different types of operators used in SQL — Arithmetic, Logical, Bitwise, and Special operators — and their purpose in writing queries.

Introduction

Operators in SQL are symbols or keywords used to perform operations on data within queries, such as calculations, comparisons, and condition-based filtering. SQL operators are broadly classified into Arithmetic, Logical, Bitwise, and Special operators, each serving a distinct purpose in data manipulation and retrieval.

1. Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations on numeric data within a query. These include addition, subtraction, multiplication, division, and modulus (remainder). They are commonly used to compute derived values such as totals, discounts, averages, or percentages directly within a SELECT statement, without needing to alter the underlying stored data.



The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL query that creates an 'employees' table and inserts five records. The right pane shows the output of a SELECT query that calculates derived values for each employee.

```
1 CREATE TABLE employees (
2   id INT,
3   name VARCHAR(50),
4   salary INT,
5   dept_id INT,
6   age INT
7 );
8
9 INSERT INTO employees VALUES
10 (1, 'Aditi', 55000, 10, 25),
11 (2, 'Bhavya', 42000, 20, 30),
12 (3, 'Kirthana', 61000, 10, 28),
13 (4, 'Geetha', 38000, 30, 22),
14 (5, 'Harshitha', 72000, NULL, 35);
15
16 SELECT
17   name,
18   salary,
19   salary * 0.10 AS ten_percent,
20   salary / 12 AS monthly_salary,
21   salary % 1000 AS remainder_check
22 FROM employees;
```

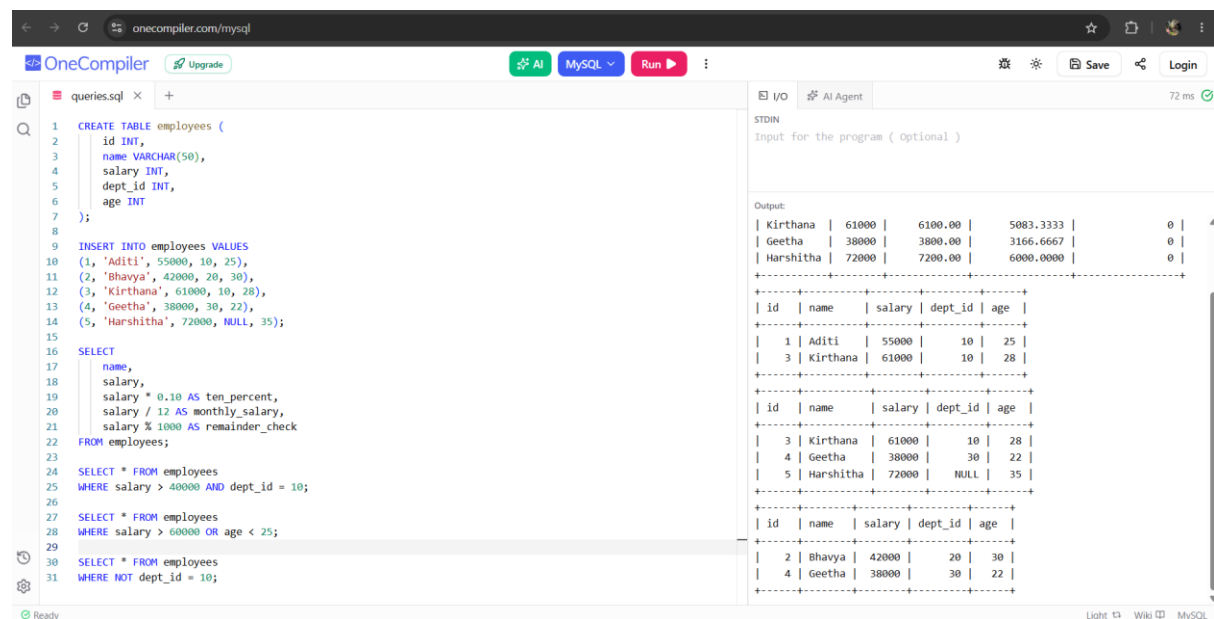
Output:

name	salary	ten_percent	monthly_salary	remainder_check
Aditi	55000	5500.00	4583.3333	0
Bhavya	42000	4200.00	3500.0000	0
Kirthana	61000	6100.00	5083.3333	0
Geetha	38000	3800.00	3166.6667	0
Harshitha	72000	7200.00	6000.0000	0

2. Logical Operators

Logical operators are used to combine multiple conditions in a WHERE clause to filter records based on more than one criterion. The main logical operators are AND, OR, and NOT. AND returns records that satisfy all specified conditions, OR returns records that satisfy at

least one condition, and NOT reverses the result of a condition. These operators allow for more precise and flexible data retrieval.



The screenshot shows the OneCompiler MySQL interface. The query editor on the left contains the following SQL code:

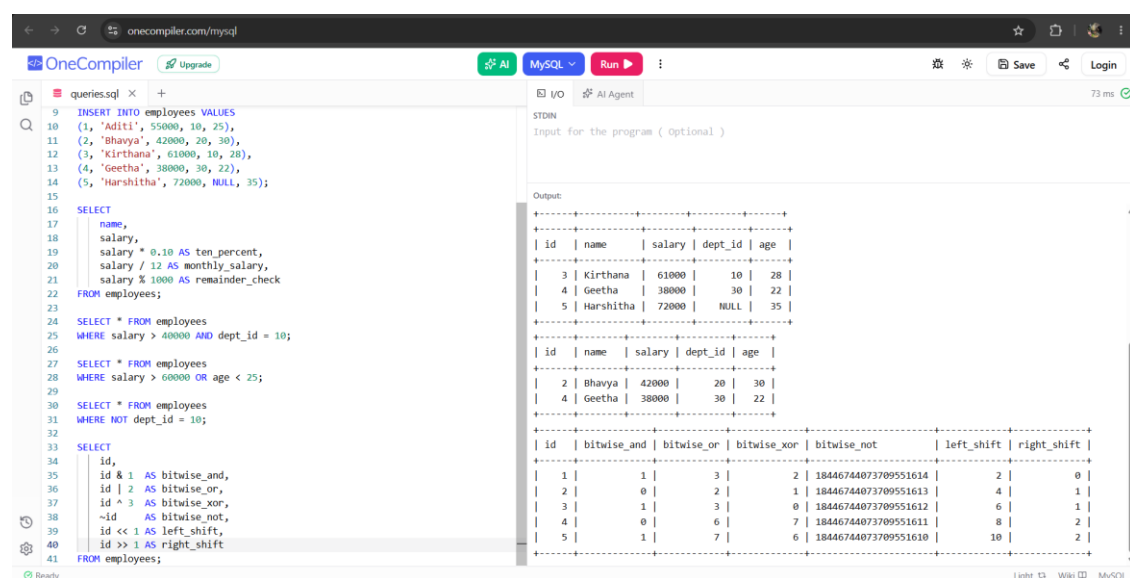
```
1 CREATE TABLE employees (  
2   id INT,  
3   name VARCHAR(50),  
4   salary INT,  
5   dept_id INT,  
6   age INT  
7 );  
8  
9 INSERT INTO employees VALUES  
10 (1, 'Aditi', 55000, 10, 25),  
11 (2, 'Bhavya', 42000, 20, 30),  
12 (3, 'Kirthana', 61000, 10, 28),  
13 (4, 'Geetha', 38000, 30, 22),  
14 (5, 'Harshitha', 72000, NULL, 35);  
15  
16 SELECT  
17   name,  
18   salary,  
19   salary * 0.10 AS ten_percent,  
20   salary / 12 AS monthly_salary,  
21   salary % 1000 AS remainder_check  
22 FROM employees;  
23  
24 SELECT * FROM employees  
25 WHERE salary > 40000 AND dept_id = 10;  
26  
27 SELECT * FROM employees  
28 WHERE salary > 60000 OR age < 25;  
29  
30 SELECT * FROM employees  
31 WHERE NOT dept_id = 10;
```

The output panel on the right shows the results of the first query (SELECT * FROM employees). The output is displayed in a table format with columns: id, name, salary, dept_id, age. The data rows are:

id	name	salary	dept_id	age
1	Aditi	55000	10	25
3	Kirthana	61000	10	28
2	Bhavya	42000	20	30
4	Geetha	38000	30	22
5	Harshitha	72000	NULL	35

3. Bitwise Operators

Bitwise operators perform operations at the binary (bit) level on integer values. They include AND, OR, XOR, NOT, and bit-shift operators. Unlike logical operators, which work on boolean conditions, bitwise operators manipulate the actual binary representation of numbers. They are typically used in specialized scenarios such as flag/permission systems, encoding, or low-level data processing, and their exact syntax can vary slightly between different database systems.



The screenshot shows the OneCompiler MySQL interface. The query editor on the left contains the following SQL code:

```
9 INSERT INTO employees VALUES  
10 (1, 'Aditi', 55000, 10, 25),  
11 (2, 'Bhavya', 42000, 20, 30),  
12 (3, 'Kirthana', 61000, 10, 28),  
13 (4, 'Geetha', 38000, 30, 22),  
14 (5, 'Harshitha', 72000, NULL, 35);  
15  
16 SELECT  
17   name,  
18   salary,  
19   salary * 0.10 AS ten_percent,  
20   salary / 12 AS monthly_salary,  
21   salary % 1000 AS remainder_check  
22 FROM employees;  
23  
24 SELECT * FROM employees  
25 WHERE salary > 40000 AND dept_id = 10;  
26  
27 SELECT * FROM employees  
28 WHERE salary > 60000 OR age < 25;  
29  
30 SELECT * FROM employees  
31 WHERE NOT dept_id = 10;  
32  
33 SELECT  
34   id,  
35   id & 1 AS bitwise_and,  
36   id | 2 AS bitwise_or,  
37   id ^ 3 AS bitwise_xor,  
38   ~id AS bitwise_not,  
39   id << 1 AS left_shift,  
40   id >> 1 AS right_shift  
41 FROM employees;
```

The output panel on the right shows the results of the last query (SELECT * FROM employees). The output is displayed in a table format with columns: id, bitwise_and, bitwise_or, bitwise_xor, bitwise_not, left_shift, right_shift. The data rows are:

id	bitwise_and	bitwise_or	bitwise_xor	bitwise_not	left_shift	right_shift
1	1	3	2	18446744073709551614	2	0
2	0	2	1	18446744073709551613	4	1
3	1	3	0	18446744073709551612	6	1
4	0	6	7	18446744073709551611	8	2
5	1	7	6	18446744073709551610	10	2

4. Special Operators

Special operators in SQL provide additional ways to filter and search data beyond basic comparisons. Common special operators include BETWEEN (to check if a value falls within a

The screenshot shows the OneCompiler web IDE interface. The left pane contains a MySQL query editor with the following code:

```

26
27 SELECT * FROM employees
28 WHERE salary > 60000 OR age < 25;
29
30 SELECT * FROM employees
31 WHERE NOT dept_id = 10;
32
33 SELECT
34     id,
35     id & 1 AS bitwise_and,
36     id | 2 AS bitwise_or,
37     id ^ 3 AS bitwise_xor,
38     ~id AS bitwise_not,
39     id << 1 AS left_shift,
40     id >> 1 AS right_shift
41 FROM employees;
42
43 SELECT * FROM employees
44 WHERE salary BETWEEN 40000 AND 65000;
45
46 SELECT * FROM employees
47 WHERE dept_id IN (10, 30);
48
49 SELECT * FROM employees
50 WHERE name LIKE 'AS';
51
52 SELECT * FROM employees
53 WHERE dept_id IS NULL;
54
55 SELECT * FROM employees e
56 WHERE EXISTS (
57     SELECT 1 FROM employees WHERE dept_id = e.dept_id AND salary > 60000
58 );

```

The right pane shows the output of the queries, which is a table with columns: id, name, salary, dept_id, age. The output is as follows:

id	name	salary	dept_id	age
1	Aditi	55000	10	25
2	Bhavya	42000	20	30
3	Kirthana	61000	10	28
1	Aditi	55000	10	25
3	Kirthana	61000	10	28
4	Geetha	38000	30	22
1	Aditi	55000	10	25
5	Harshitha	72000	NULL	35

Understanding the different categories of SQL operators — Arithmetic, Logical, Bitwise, and Special — is essential for writing efficient and accurate queries. Each type serves a specific purpose, from performing calculations to filtering, comparing, and searching data, making them fundamental building blocks of SQL programming.